AUTOMATIZACIÓN DE AUTOCLAVE

Control de Ciclo con PIC18F45K22

Electrónica Integrada
Primavera 2026

Integrantes del equipo:
Luis Alexis Calvillo Deveze

Fecha de entrega: 21-04-2026

1. Diagrama de Flujo / Pseudocódigo

El sistema implementa una máquina de estados finitos con cuatro estados: CONFIG, CORRIENDO, PAUSA y TERMINADO. El Timer0 genera interrupciones cada 200 ms que actualizan la cuenta regresiva y la bandera de parpadeo. A continuación se presenta el pseudocódigo completo del algoritmo:

```
INICIO
  Inicializar puertos (TRISA=entrada, TRISB/TRISC=salida)
  Configurar Timer0 con recarga 0x9E58 (periodo ~200 ms)
  Habilitar interrupciones globales

  BUCLE PRINCIPAL:
    SI estado == CONFIG ENTONCES
      Apagar LEDs y BUZZER
      Leer SW_SRV, SW_1, SW_2 → asignar tiempo_restante
      Mostrar tiempo_restante en display 7-seg
      SI BTN_START presionado Y tiempo > 0 ENTONCES
        Esperar anti-rebote (20 ms)
        estado = CORRIENDO
      FIN SI

    SI estado == CORRIENDO ENTONCES
      Mostrar tiempo_restante en display
      SI tiempo_restante <= 2 ENTONCES LED_VERDE = parpadeo
      SINO LED_VERDE = ENCENDIDO
      SI tiempo_restante == 0 ENTONCES estado = TERMINADO
      SI BTN_PR presionado ENTONCES estado = PAUSA

    SI estado == PAUSA ENTONCES
      LED_AMARILLO = parpadeo
      SI BTN_PR presionado ENTONCES estado = CORRIENDO

    SI estado == TERMINADO ENTONCES
      LED_ROJO = ENCENDIDO
      BUZZER = ENCENDIDO
      Display = 0

    SI BTN_CLR presionado Y estado != CONFIG ENTONCES
      estado = CONFIG
      Reiniciar variables y salidas
  FIN BUCLE
FIN

── ISR Timer0 (cada ~200 ms) ──
  Recargar Timer0
  Alternar bandera_parpadeo
  SI estado == CORRIENDO ENTONCES
    contador_200ms++
    SI contador_200ms >= 5 ENTONCES  // 1 segundo
      contador_200ms = 0
      SI tiempo_restante > 0 ENTONCES tiempo_restante--
```

```
  FIN SI
FIN SI
```

Descripción de estados

- ESTADO_CONFIG (0): El sistema espera la configuración de tiempo mediante los switches SW_1, SW_2 y SW_SRV. El display muestra el tiempo seleccionado. Al presionar BTN_START (activo-bajo) se inicia el ciclo.
- ESTADO_CORRIENDO (1): La cuenta regresiva activa. LED verde encendido; parpadea en los últimos 2 segundos. Al llegar a 0 transiciona a TERMINADO. BTN_PR pausa el ciclo.
- ESTADO_PAUSA (2): Tiempo congelado. LED amarillo parpadea. BTN_PR reanuda.
- ESTADO_TERMINADO (3): LED rojo fijo y buzzer activo. Solo BTN_CLR devuelve al sistema a CONFIG.

2. Programa en Lenguaje C

El siguiente código fue desarrollado para MPLAB X IDE v6.30 con compilador XC8 v3.10, dirigido al microcontrolador PIC18F45K22 operando a 8 MHz con oscilador interno. Se implementa la máquina de estados descrita en la sección anterior.

2.1 Correcciones aplicadas respecto al prototipo inicial

- Lógica de botones BTN_PR y BTN_CLR corregida a activo-bajo (pull-ups de 10 kΩ en el esquemático). Las comparaciones cambiaron de == 1 a == 0 y los bucles de espera de while(BTN == 1) a while(BTN == 0).
- Buzzer activado en ESTADO_TERMINADO. En la versión original BUZZER permanecía en 0, impidiendo la alerta sonora al concluir el ciclo.

2.2 Código fuente documentado

```
#include <xc.h>
#define _XTAL_FREQ 8000000

#pragma config FOSC = INTIO67
#pragma config WDTEN = OFF
#pragma config MCLRE = INTMCLR

// Entradas
#define BTN_START   PORTAbits.RA0
#define BTN_PR      PORTAbits.RA1
#define BTN_CLR     PORTAbits.RA2
#define SW_1        PORTAbits.RA3
#define SW_2        PORTAbits.RA4
#define SW_SRV      PORTAbits.RA5

// Salidas
#define BUZZER        LATBbits.LATB4
#define LED_VERDE     LATBbits.LATB5
#define LED_AMARILLO  LATBbits.LATB6
#define LED_ROJO      LATBbits.LATB7
#define DISPLAY_7S    LATC

// Preload Timer0
#define TMR0_PRELOAD_H  0x9E
#define TMR0_PRELOAD_L  0x58

// Estados del sistema
#define ESTADO_CONFIG      0
#define ESTADO_CORRIENDO   1
#define ESTADO_PAUSA       2
#define ESTADO_TERMINADO   3

// Tabla 7 segmentos (0-9)
```

```
unsigned char tabla_7seg[10] = {
    0x3F, 0x06, 0x5B, 0x4F, 0x66,
    0x6D, 0x7D, 0x07, 0x7F, 0x67
};

// Variables compartidas con ISR
volatile unsigned char contador_200ms = 0;
volatile unsigned char bandera_parpadeo = 0;
volatile unsigned char tiempo_restante = 0;
volatile unsigned char estado = ESTADO_CONFIG;

// Tiempos de ciclo (segundos)
const unsigned char ciclo_fast       = 3;
const unsigned char ciclo_standard   = 5;
const unsigned char ciclo_prolongado = 9;

// ── ISR ──────────────────────────────────────────────────────
void __interrupt() My_ISR(void) {
    if (INTCONbits.TMR0IF) {
        TMR0H = TMR0_PRELOAD_H;
        TMR0L = TMR0_PRELOAD_L;
        bandera_parpadeo = !bandera_parpadeo;
        if (estado == ESTADO_CORRIENDO) {
            if (++contador_200ms >= 5) {
                contador_200ms = 0;
                if (tiempo_restante > 0) tiempo_restante--;
            }
        }
        INTCONbits.TMR0IF = 0;
    }
}

// ── main ─────────────────────────────────────────────────────
void main(void) {
    OSCCONbits.IRCF = 0b110;  // 8 MHz
    ANSELA = ANSELB = ANSELC = 0x00;
    TRISA = 0xFF; TRISB = 0x00; TRISC = 0x00;
    LED_VERDE = LED_AMARILLO = LED_ROJO = BUZZER = 0;
    DISPLAY_7S = tabla_7seg[0];

    T0CON = 0x03;
    TMR0H = TMR0_PRELOAD_H; TMR0L = TMR0_PRELOAD_L;
    INTCONbits.TMR0IF = 0;
    INTCONbits.TMR0IE = INTCONbits.GIE = 1;
    T0CONbits.TMR0ON = 1;

    while (1) {
        // ESTADO 0 - CONFIG
        if (estado == ESTADO_CONFIG) {
            LED_VERDE = LED_AMARILLO = LED_ROJO = BUZZER = 0;
            if (SW_SRV) {
                if      (SW_1 && !SW_2) tiempo_restante = 1;
                else if (!SW_1 && SW_2) tiempo_restante = 2;
                else if (SW_1 && SW_2)  tiempo_restante = 3;
                else                    tiempo_restante = 0;
            } else {
                if      (SW_1 && !SW_2) tiempo_restante = ciclo_fast;
                else if (!SW_1 && SW_2) tiempo_restante = ciclo_standard;
                else if (SW_1 && SW_2)  tiempo_restante = ciclo_prolongado;
                else                    tiempo_restante = 0;
            }
```

```
            DISPLAY_7S = tabla_7seg[tiempo_restante];
            if (BTN_START == 0 && tiempo_restante > 0) {
                __delay_ms(20);
                if (BTN_START == 0) {
                    estado = ESTADO_CORRIENDO;
                    contador_200ms = 0;
                    while (BTN_START == 0);
                    __delay_ms(20);
                }
            }
        }
        // ESTADO 1 - CORRIENDO
        else if (estado == ESTADO_CORRIENDO) {
            DISPLAY_7S = tabla_7seg[tiempo_restante];
            LED_AMARILLO = LED_ROJO = BUZZER = 0;
            LED_VERDE = (tiempo_restante <= 2) ? bandera_parpadeo : 1;
            if (tiempo_restante == 0) estado = ESTADO_TERMINADO;
            if (BTN_PR == 0) {
                __delay_ms(20);
                if (BTN_PR == 0) {
                    estado = ESTADO_PAUSA;
                    while (BTN_PR == 0);
                    __delay_ms(20);
                }
            }
        }
        // ESTADO 2 - PAUSA
        else if (estado == ESTADO_PAUSA) {
            LED_VERDE = LED_ROJO = BUZZER = 0;
            LED_AMARILLO = bandera_parpadeo;
            if (BTN_PR == 0) {
                __delay_ms(20);
                if (BTN_PR == 0) {
                    estado = ESTADO_CORRIENDO;
                    contador_200ms = 0;
                    while (BTN_PR == 0);
                    __delay_ms(20);
                }
            }
        }
        // ESTADO 3 - TERMINADO
        else if (estado == ESTADO_TERMINADO) {
            LED_VERDE = LED_AMARILLO = 0;
            LED_ROJO = BUZZER = 1;
            DISPLAY_7S = tabla_7seg[0];
        }
        // CANCELACION GLOBAL (activo-bajo)
        if (BTN_CLR == 0 && estado != ESTADO_CONFIG) {
            __delay_ms(20);
            if (BTN_CLR == 0) {
                estado = ESTADO_CONFIG;
                tiempo_restante = contador_200ms = bandera_parpadeo = 0;
                LED_VERDE = LED_AMARILLO = LED_ROJO = BUZZER = 0;
                DISPLAY_7S = tabla_7seg[0];
                while (BTN_CLR == 0);
                __delay_ms(20);
            }
        }
    }
}
```

3. Diagrama Esquemático en PROTEUS

3.1 Descripción del circuito

El circuito fue diseñado y simulado en PROTEUS 8. Los componentes principales son:

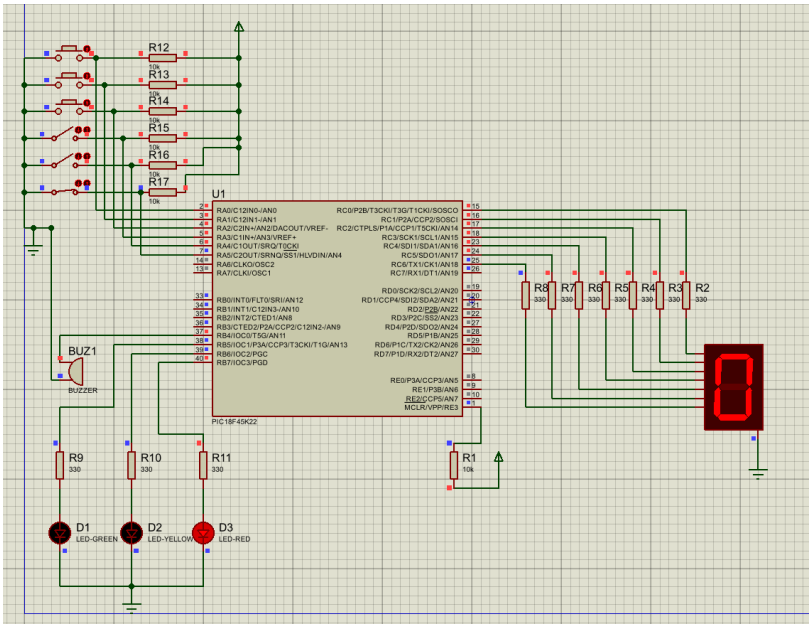
- PIC18F45K22 como unidad de procesamiento central.
- 6 entradas digitales (RA0–RA5): 3 pulsadores (BTN_START, BTN_PR, BTN_CLR) y 3 switches (SW_1, SW_2, SW_SRV). Cada entrada cuenta con una resistencia de pull-up de 10 kΩ a VCC. Al presionar, el pin baja a GND (lógica activo-bajo).
- Display de 7 segmentos de cátodo común conectado a PORTC a través de resistencias limitadoras de 330 Ω (R2–R8).
- 3 LEDs (verde, amarillo, rojo) en RB5, RB6 y RB7, respectivamente, con resistencias de 330 Ω (R9–R11).
- Buzzer piezoeléctrico en RB4 con resistencia de 330 Ω (R1 = 10 kΩ de pull-up en MCLR).

3.2 Mapa de pines

Puerto A (entradas): RA0 = BTN_START, RA1 = BTN_PR, RA2 = BTN_CLR, RA3 = SW_1, RA4 = SW_2, RA5 = SW_SRV.

Puerto B (salidas): RB4 = BUZZER, RB5 = LED_VERDE, RB6 = LED_AMARILLO, RB7 = LED_ROJO.

Puerto C (salida): RC0–RC6 = segmentos a–g del display de 7 segmentos (cátodo común).



4. Evidencias de Pruebas

Las pruebas de funcionamiento se realizaron en el simulador PROTEUS y en hardware real. Se verificaron los siguientes escenarios:

- Selección correcta de tiempo según combinación de switches en modo normal y modo servicio (SW_SRV = 1).
- Inicio de ciclo con BTN_START: transición CONFIG → CORRIENDO y cuenta regresiva en display.
- Parpadeo de LED verde en los últimos 2 segundos del ciclo.
- Pausa y reanudación con BTN_PR: LED amarillo parpadeando en pausa, display congelado.
- Finalización de ciclo: LED rojo fijo, buzzer activo, display en 0.
- Cancelación global con BTN_CLR desde cualquier estado distinto a CONFIG.

Video de demostración:

https://youtu.be/zior4u__iR0

5. Discusión

5.1 Desarrollo del programa

El diseño centrado en una máquina de estados finitos permitió separar claramente la lógica de cada fase del ciclo del autoclave, facilitando la depuración y la adición de nuevas funcionalidades. El uso del Timer0 con interrupciones garantiza que la cuenta regresiva sea independiente de la lógica del bucle principal, evitando bloqueos por esperas activas prolongadas.

La detección de botones activos-bajos con pull-ups externos es una práctica robusta que reduce el ruido y garantiza estados bien definidos. El anti-rebote por software (retardo de 20 ms + verificación doble) evita transiciones involuntarias de estado.

5.2 Desempeño del equipo

El equipo distribuyó las tareas de forma equitativa: diseño del circuito en PROTEUS, desarrollo del algoritmo en pseudocódigo, implementación en C y pruebas de validación. La comunicación efectiva y la revisión cruzada de código permitieron identificar los errores de lógica (botones activo-bajo y buzzer) antes de la entrega final.

5.3 Objetivos de aprendizaje alcanzados

- Implementación de máquinas de estados finitos en sistemas embebidos.
- Configuración y uso de Timer0 con interrupciones en PIC18F45K22.
- Diseño de circuitos de interfaz: pull-ups, LEDs, display de 7 segmentos y buzzer.
- Técnicas de anti-rebote por software para pulsadores.
- Simulación y validación de circuitos digitales con PROTEUS.
- Uso del entorno MPLAB X IDE con compilador XC8.

6. Conclusiones

El proyecto permitió desarrollar un sistema de temporización para autoclave funcional, con control de estados bien definidos y retroalimentación visual y sonora para el operador. La combinación de hardware simple (PIC18F45K22, LEDs, display) con una arquitectura de software basada en interrupciones demuestra cómo es posible implementar sistemas de control confiables con recursos limitados.

La corrección de los errores detectados (lógica activo-bajo y activación del buzzer) resaltó la importancia de relacionar siempre el código con el esquemático físico, ya que una diferencia de nivel lógico puede inutilizar por completo una función del sistema.

Como trabajo futuro se podría agregar un display de múltiples dígitos para mostrar tiempos mayores, una etapa de potencia para controlar la válvula y la resistencia calefactora reales, y comunicación serial para registro de ciclos.

7. Bibliografía

- Microchip Technology. (2010). PIC18F45K22 Data Sheet. Recuperado de https://ww1.microchip.com/downloads/en/DeviceDoc/40001412F.pdf
- Microchip Technology. (2024). MPLAB XC8 C Compiler User's Guide. Recuperado de https://www.microchip.com/mplab/compilers
- Labcenter Electronics. (2023). PROTEUS VSM User Manual. Recuperado de https://www.labcenter.com
- Mazidi, M. A., McKinlay, R. D., & Causey, D. (2008). PIC Microcontroller and Embedded Systems. Pearson Prentice Hall.
- Reyes, A. F. (2006). Microcontroladores PIC: Diseño Práctico de Aplicaciones. McGraw-Hill.